

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      stm32f1xx_it.c
 * @brief     Interrupt Service Routines.
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 * *****
 */
/* USER CODE END Header */

/* Includes -----*/
#include "main.h"
#include "stm32f1xx_it.h"
/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "stm32f1xx_ll_utils.h"
#include "stm32f1xx_ll_exti.h"

// FIX: Declare the function prototype so stm32f1xx_it.c knows it exists
void UART_Send_String(const char *str);

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN TD */

/* USER CODE END TD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
/* USER CODE BEGIN PV */

extern __IO uint8_t currentMode; // Your existing variable
static __IO uint32_t last_press_time = 0; // Timestamp of the last valid button press

/* USART RX buffer definitions */
volatile char rx_buffer[RX_BUF_SIZE];
volatile uint8_t rx_index = 0;

volatile uint8_t command_ready = 0; // Flag: 1 means the command line is fully received

/* USER CODE END PV */

/* Private function prototypes -----*/
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

```

```

/* External variables -----*/

/* USER CODE BEGIN EV */

extern volatile uint32_t ms_ticks;
uint32_t LL_GetTick(void); // Prototype of the tick getter function

/* USER CODE END EV */

/*****
 *      Cortex-M3 Processor Interruption and Exception Handlers      */
*****/
/**
 * @brief This function handles Non maskable interrupt.
 */
void NMI_Handler(void)
{
    /* USER CODE BEGIN NonMaskableInt_IRQn 0 */

    /* USER CODE END NonMaskableInt_IRQn 0 */
    /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
    while (1)
    {
    }
    /* USER CODE END NonMaskableInt_IRQn 1 */
}

/**
 * @brief This function handles Hard fault interrupt.
 */
void HardFault_Handler(void)
{
    /* USER CODE BEGIN HardFault_IRQn 0 */

    /* USER CODE END HardFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_HardFault_IRQn 0 */
        /* USER CODE END W1_HardFault_IRQn 0 */
    }
}

/**
 * @brief This function handles Memory management fault.
 */
void MemManage_Handler(void)
{
    /* USER CODE BEGIN MemoryManagement_IRQn 0 */

    /* USER CODE END MemoryManagement_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_MemoryManagement_IRQn 0 */
        /* USER CODE END W1_MemoryManagement_IRQn 0 */
    }
}

/**
 * @brief This function handles Prefetch fault, memory access fault.
 */
void BusFault_Handler(void)
{
    /* USER CODE BEGIN BusFault_IRQn 0 */

    /* USER CODE END BusFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_BusFault_IRQn 0 */
        /* USER CODE END W1_BusFault_IRQn 0 */
    }
}

```

```

/**
 * @brief This function handles Undefined instruction or illegal state.
 */
void UsageFault_Handler(void)
{
    /* USER CODE BEGIN UsageFault_IRQn 0 */

    /* USER CODE END UsageFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_UsageFault_IRQn 0 */
        /* USER CODE END W1_UsageFault_IRQn 0 */
    }
}

/**
 * @brief This function handles System service call via SWI instruction.
 */
void SVC_Handler(void)
{
    /* USER CODE BEGIN SVC_IRQn 0 */

    /* USER CODE END SVC_IRQn 0 */
    /* USER CODE BEGIN SVC_IRQn 1 */

    /* USER CODE END SVC_IRQn 1 */
}

/**
 * @brief This function handles Debug monitor.
 */
void DebugMon_Handler(void)
{
    /* USER CODE BEGIN DebugMonitor_IRQn 0 */

    /* USER CODE END DebugMonitor_IRQn 0 */
    /* USER CODE BEGIN DebugMonitor_IRQn 1 */

    /* USER CODE END DebugMonitor_IRQn 1 */
}

/**
 * @brief This function handles Pendable request for system service.
 */
void PendSV_Handler(void)
{
    /* USER CODE BEGIN PendSV_IRQn 0 */

    /* USER CODE END PendSV_IRQn 0 */
    /* USER CODE BEGIN PendSV_IRQn 1 */

    /* USER CODE END PendSV_IRQn 1 */
}

/**
 * @brief This function handles System tick timer.
 */
void SysTick_Handler(void)
{
    /* USER CODE BEGIN SysTick_IRQn 0 */
    ms_ticks++; // Increment the global millisecond counter every 1 ms
    /* USER CODE END SysTick_IRQn 0 */

    /* USER CODE BEGIN SysTick_IRQn 1 */

    /* USER CODE END SysTick_IRQn 1 */
}

/*****
 * STM32F1xx Peripheral Interrupt Handlers
 */

```

```

/* Add here the Interrupt Handlers for the used peripherals.                  */
/* For the available peripheral interrupt handler names,                      */
/* please refer to the startup file (startup_stm32f1xx.s).                  */
/*******/

/* USER CODE BEGIN 1 */
// Link to the global variable defined in main.c
extern __IO uint8_t currentMode;

/**

* @brief This function handles USART1 global interrupt.
*/
void USART1_IRQHandler(void)
{
    /* USER CODE BEGIN USART1_IRQn 0 */

    /* Check if the RXNE interrupt flag is active and interrupt generation is enabled */
    if (LL_USART_IsActiveFlag_RXNE(USART1) && LL_USART_IsEnabledIT_RXNE(USART1))
    {
        /* Reading the data register automatically clears the RXNE flag */
        char received_char = (char)LL_USART_ReceiveData8(USART1);

        /* Echo feature: send the received character back to the transmitter host */
        while (!LL_USART_IsActiveFlag_TXE(USART1));
        LL_USART_TransmitData8(USART1, received_char);

        /* Process characters only if the previous command has been evaluated in main */
        if (command_ready == 0)
        {
            /* Check if the received character is a valid command terminator */
            if (received_char == '\n' || received_char == '\r')
            {
                if (rx_index > 0) /* Ensure the buffer actually contains command letters */
                {
                    rx_buffer[rx_index] = '\0'; /* Terminate the string safely */
                    command_ready = 1;          /* Notify the main loop that data is ready */
                }
            }
            else
            {
                /* Ignore any unexpected leading control characters (like extra \r or \n) */
                if (received_char >= 32 && received_char <= 126)
                {
                    if (rx_index < (RX_BUF_SIZE - 1))
                    {
                        rx_buffer[rx_index++] = received_char;
                    }
                    else
                    {
                        rx_index = 0; /* Reset index on buffer overflow constraint */
                    }
                }
            }
        }
    }
    /* USER CODE END USART1_IRQn 0 */
}

/**

* @brief This function handles EXTI Line 10 to 15 interrupts (Triggers on PB12 press).
*/
void EXTI15_10_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI15_10_IRQn 0 */

    /* Check if the interrupt was actually triggered by EXTI Line 12 (PB12)
    if (LL_EXTI_IsActiveFlag_0_31(LL_EXTI_LINE_12) != RESET)
    {

```

```

// Clear the interrupt pending flag IMMEDIATELY to avoid infinite looping
LL_EXTI_ClearFlag_0_31(LL_EXTI_LINE_12);

// Get the current system uptime in milliseconds from SysTick
uint32_t current_time = LL_GetTick();

// Software debounce: check if 500 ms have passed since the last valid press
if ((current_time - last_press_time) > 500)
{
    // If 500 ms passed, it is a valid button press, not a contact bounce

    // Advance to the next operation mode
    currentMode++;
    if (currentMode > TOTAL_MODES)
    {
        currentMode = MODE_LED_OFF;
    }

    // 6. Transmit the current status log via USART
    switch (currentMode)
    {
        case MODE_LED_OFF:  UART_Send_String("Mode Changed: LED OFF\r\n"); break;
        case MODE_LED_ON:   UART_Send_String("Mode Changed: LED ON\r\n");  break;
        case MODE_LED_SOS:  UART_Send_String("Mode Changed: SOS \r\n");    break;
        case MODE_LED_HTB:  UART_Send_String("Mode Changed: HEARTBEAT\r\n"); break;
        default: break;
    }

    last_press_time = current_time; // Save the timestamp of the current valid press
}
}
/* USER CODE END EXTI15_10_IRQn 0 */
}
/* USER CODE END 1 */

```